ACS MILESTONE 31

System Reference

A top-down technical reference for how ACS is organized, how its domains fit together, how publishing works, and how the future AI-agnostic provider model is intended to be integrated.

Table of Contents

1. Purpose
2. Executive Summary
3. Product Model
4. Core Domains
5. Domain Boundaries
6. System Architecture
7. Data Model Hierarchy
8. Application Modes
9. End-To-End Workflows
10. Publishing In Detail
11. AI-Agnostic Integration In Detail
12. Runtime Behavior Model
13. Editor Behavior Model
14. Presentation And Skinning
15. Persistence And Storage
16. Validation And Quality Gates
17. Current Limitations
18. Future Roadmap Alignment
19. Subject Glossary
20. Technical Catalog
21. Diagrams Appendix
22. Change Log And Reference Maintenance

Purpose

This document explains the system in reader-first order. It is meant for developers, maintainers, advanced designers, and future integrators who need to understand how the application fits together, where responsibilities live, and how future work is expected to plug into the existing architecture.

The User Guide explains how to use the application. This System Reference explains how the application works.

Executive Summary

ACS is a browser-based adventure construction environment built around a shared authored-content model, a renderer-agnostic runtime, a browser editor, a release-backed publishing system, and a future AI-agnostic review boundary. The single most important architectural rule is separation: rules are separate from presentation, editor behavior is separate from editor presentation, authored content is separate from runtime state, mutable drafts are separate from immutable releases, and AI provider details are separate from the application's stable review lifecycle.

This separation is what makes future renderer families, editor skins, richer publishing modes, AI-assisted authoring, multiplayer, and mobile shells possible without forcing a full architectural rewrite.

Product Model

What The Application Is

The application is three things at once: a playable adventure runtime, a browser-based construction set, and a release/export system. Those three pieces share one content model instead of each inventing a private data shape.

Who It Is For

- **Player:** plays adventures, saves progress, and loads sessions
- **Designer:** edits adventures, validates drafts, playtests, publishes, and exports
- **Reviewer:** inspects release metadata and handoff artifacts
- **Future integrator:** adds providers, renderers, or distribution shells

Main Modes

- **Play Mode:** browser runtime
- **Edit Mode:** browser editor
- **Test & Publish:** release preparation, export preview, and handoff

Main Artifacts

- **Draft:** mutable working state
- **Project:** mutable backend-backed draft record
- **Release:** immutable published snapshot
- **Forkable package:** editable handoff for another designer
- **Standalone package:** play-only handoff for another player
- **Review package:** reviewer-facing release summary bundle
- **AI review/import artifacts:** portable future AI handoff layers

---

Core Domains

Adventure Domain

This is the authored world: metadata, maps, regions, tiles, definitions, instances, dialogue, quests, items, cues, visuals, and library content.

Runtime Domain

This is live game state and gameplay behavior: movement, interaction, flags, inventory, trigger execution, dialogue state, enemy cadence, and event emission.

Editor Domain

This is the authoring workflow layer: draft mutation, workspace behavior, diagnostics, validation presentation, and project/release orchestration.

Publishing Domain

This is the release-backed export layer: forkable, standalone, handoff, integrity, and review artifacts.

AI Domain

This is the future provider-agnostic request/review/apply boundary: provider manifests, requests, proposals, review reports, apply plans, and portable AI handoff objects.

Domain Boundaries

Package / App	Responsibility
`packages/domain`	Shared authored-content types
`packages/content-schema`	Normalization and compatibility cleanup
`packages/runtime-core`	Rules, state, commands, triggers, and events
`packages/runtime-2d`	Canvas gameplay presentation
`packages/editor-core`	Pure draft mutations and authoring-side reports
`packages/validation`	Shared structural and reference validation
`packages/persistence`	Runtime snapshot persistence wrappers
`packages/project-api`	Shared project and release contract
`packages/publishing`	Release-backed export generation
`packages/default-content`	Built-in starter content helpers
`packages/ai-core`	Provider-agnostic AI request/review/apply/import/export contracts
`apps/web`	Browser runtime and editor presentation
`apps/api`	Local project/release API and standalone bundle assembly

Boundary rule: vendor SDKs, browser layout logic, and distribution shell details should not leak into the shared packages that define content, rules, validation, or AI review contracts.

System Architecture

```
AdventurePackage
  -> content-schema normalization
  -> runtime-core for gameplay rules
  -> editor-core for authoring mutations
  -> validation for structural checks
  -> publishing for release-backed exports
  -> ai-core for future reviewed AI handoffs

runtime-core snapshot
  -> runtime-2d renderer
  -> browser runtime UI

editor-core draft
  -> browser editor UI
  -> project/release API
```

In practice, this means authored content is normalized once, then reused by runtime, editor, validation, publishing, and future AI review layers instead of being copied into separate application-specific data models.

Data Model Hierarchy

Authored Data Vs Runtime State

`AdventurePackage` is the authored world. Runtime session state is what is happening right now in play. One authored package can produce many different runtime sessions.

Definitions Vs Instances

Reusable definitions describe what a thing is. Instances describe where a particular copy is placed and any local override state it needs.

Mutable Vs Immutable Artifacts

Drafts and projects are mutable. Releases and export artifacts are immutable. This keeps distribution and review reproducible.

Application Modes

Play Mode

Loads a sample adventure, a local draft playtest, or a published release; creates a runtime session; accepts commands; advances gameplay state; and renders the result.

Edit Mode

Loads a draft and lets the designer change metadata, world structure, maps, libraries, logic, validation state, and project/release state.

Test & Publish

Handles validation, diagnostics, project persistence, release publication, artifact previews, and exports.

End-To-End Workflows

Runtime Command Flow

```
Player input
  -> apps/web runtime handler
  -> runtime-core command dispatch
  -> state mutation and engine events
  -> browser UI updates
  -> runtime-2d render from snapshot
```

Editor Mutation Flow

```
Designer action
  -> apps/web editor handler
  -> editor-core pure mutation helper
  -> updated draft
  -> validation and diagnostics
  -> rerender current workspace
```

Project And Release Flow

```
Draft
  -> validate
  -> create/save project
  -> publish immutable release
  -> preview/export release-backed artifacts
```

Publishing In Detail

Publishing is intentionally release-backed. Designers do not export directly from arbitrary draft state. They edit a draft, save or create a project, publish an immutable release, and derive all shareable outputs from that release.

Why This Matters

- exports become reproducible
- release metadata has one stable source
- reviewers can compare export modes reliably
- editable and play-only handoffs stay clearly separated

Forkable Package

The editable handoff for another designer. It preserves authored content, provenance, and import guidance.

Standalone Package

The play-only handoff for players. It is a static runtime web bundle with distribution metadata, handoff guidance, and optional launcher helpers.

Review Package

The reviewer-facing handoff. It bundles the release handoff summary and artifact integrity report into one package for external inspection.

Delivery Model

The standalone package remains the same static web bundle regardless of delivery shell. Manual hosting, local launcher, hosted deployment, and later desktop wrappers are all delivery modes over the

same underlying package.

AI-Agnostic Integration In Detail

This is the section that matters most for future provider integration, so it is intentionally more detailed than the average feature explanation.

The Problem The AI Layer Is Solving

If AI support is added carelessly, the application gets tied to one SDK, one vendor response shape, and one UI flow. It also becomes easy for model output to bypass structured content rules and mutate editor state directly. The AI-agnostic layer exists to prevent that.

The Core Rule

AI is allowed to propose structured content. AI is not allowed to directly mutate the authored world.

What `@acs/ai-core` Actually Is

`packages/ai-core` is not a live model client and not an agent loop. It is the application's stable AI language and review protocol. It defines what the application means by provider, request, proposal, review, apply readiness, and portable AI handoff.

What `@acs/ai-core` Is Not

- not OpenAI-specific
- not Anthropic-specific
- not local-model-specific
- not UI-specific
- not browser-specific

The Intended AI Lifecycle

1. Choose a provider
2. Build a normalized generation request
3. Pass that request through a provider-specific adapter
4. Receive the provider response
5. Translate that response into a stable proposal envelope
6. Validate the request and proposal

7. Build review artifacts
8. Have a human accept or reject the proposal
9. If accepted, build an application plan
10. Only then apply it through controlled authoring mutations

Main AI-Core Types

`AiProviderManifest`

The stable identity card for a provider. It describes what the provider is and what capabilities it offers from the application's perspective.

`AdventureGenerationRequest`

The normalized application-level generation request. It expresses the app's intent before any vendor-specific translation happens.

`AiAdventureProposal`

The structured proposal envelope returned for review. This is not supposed to be "whatever the vendor returned." It is the vendor response translated back into stable application terms.

`AdventureGenerationPlan`

The explicit review-first step sequence for generation, validation, review, and apply readiness.

`AiProposalReviewReport`

The shared readiness report for a proposal. It answers whether the proposal is blocked, reviewable, or ready for the next step.

`AiGenerationSessionRecord`

The portable session bundle for request, plan, proposal, and review state. It prevents later UI layers from scattering those links in browser-only state.

`AiProposalChangeSummary`

The impact summary that explains what the proposal would change before a human accepts it.

`AiProposalApplicationPlan`

The safe-apply bridge between review and controlled mutation. It explains whether a proposal can be applied and what sections would be affected.

Why There Are So Many Review, Bundle, Archive, And Dossier Types

These layers are not just redundant wrappers. They exist so reviewed AI work can be stored, exported, re-imported, audited, and shared without reconstructing it from temporary UI state. They are the transport and audit layers for the AI review lifecycle, and they now include both handoff integrity reports and import dossier integrity reports so later UI or API layers can judge readiness from one provider-agnostic status object.

What A Provider Adapter Should Do

A provider adapter should live outside `ai-core`. Its job should be:

1. accept a stable `AdventureGenerationRequest`
2. translate it into the vendor's request shape
3. call the vendor SDK or HTTP API
4. receive the vendor response
5. translate that response into a stable `AiAdventureProposal`
6. return the proposal to the shared review flow

That means the adapter is primarily a translator and transport layer. The application's review semantics stay shared.

What A Provider Adapter Must Not Do

- it must not mutate editor state directly
- it must not bypass validation
- it must not bypass review reports
- it must not decide on its own that a proposal is acceptable
- it must not call low-level authoring mutations outside the reviewed apply path

How To Implement A New Provider Later

1. Define a new `AiProviderManifest`.
2. Add that provider to the registry.
3. Write an adapter that converts:
   - `AdventureGenerationRequest` to the vendor request
   - vendor response to `AiAdventureProposal`
4. Pass the proposal through validation, review report creation, impact summary creation, and application planning.

5. Expose that reviewed lifecycle in UI or API.

The first live-provider rollout should do this with one complete adapter end to end rather than several partial adapters at once. The goal is to prove that a real model can be configured, called, reviewed, and applied through the same lifecycle the package-level contracts already define.

How To Switch Between AI Models Or Vendors

This is one of the main reasons the architecture exists. Switching providers should not require rewriting the editor, runtime, content model, or review lifecycle.

What changes:

- provider manifest choice
- provider adapter implementation
- vendor-specific limits and warnings

What stays stable:

- generation request shape
- proposal shape
- review report shape
- session record shape
- change summary shape
- application plan shape
- portable review/import handoff layers
- human review requirement

That is what makes the layer provider-agnostic rather than just “vendor-lightly-coupled.”

What Exists Today Vs What Is Still Missing

Exists today: the shared contracts, portable review/import layers, and Milestone 32A product-level request planning for creating a new game, finishing an existing game, or expanding an existing game from an AI prompt.

Still missing: live provider adapters, end-user AI controls, credential management, actual request execution UI, and reviewed apply UI. The seam is built; the concrete provider connections and user-facing workflows come later.

Runtime Behavior Model

The runtime is responsible for gameplay meaning, not presentation chrome. It owns movement, interaction, inspection, trigger execution, dialogue state, quest progression, enemy cadence, and runtime event emission.

Current behavior is still mostly player-centered. Future actor-capable action work should extend these pathways rather than bypass them. Large-map presentation is also still simpler than the final target: oversized maps are allowed in authored content, but true player-facing map-window scrolling is still future presentation work.

Editor Behavior Model

The editor is responsible for authoring workflows, not core gameplay rules. The long-term goal is an editor that can be reskinned or reorganized visually without forking authoring behavior.

- current workspace should drive what controls are visible
- mutations should go through `editor-core` where feasible
- validation should use shared package rules
- diagnostics should summarize authoring quality without replacing validation
- the same draft model should survive future editor skins

Presentation And Skinning

Gameplay rendering and editor presentation are both intended to become skinnable later because their behavior boundaries are being kept separate now.

Gameplay Presentation

- `runtime-core` owns gameplay meaning
- `runtime-2d` and later renderer families own visual presentation

The same separation should be used for oversized-map scrolling: the engine should continue to think in map coordinates, while renderers decide how to frame large maps.

Editor Presentation

- shared draft state and editor-core helpers should remain stable

- future WorldTree or branded shells should sit over those same contracts

The live UI surface registry is tracked in `docs/ux-skinning-inventory.md` and `docs/ux-skinning-inventory.json`.

Persistence And Storage

Browser Storage

Stores runtime saves, local drafts, and the remembered active project id.

Local API Storage

Stores mutable projects and immutable releases. The current backing store is `apps/api/data/store.json`.

Export Artifacts

Store release-backed forkable, standalone, reviewer, and future AI review/import handoffs.

Validation And Quality Gates

Quality is treated as a layered gate.

- `npm run quality`
- `npm test`
- `npm run docs:validate`

Together these cover complexity, typecheck, unit tests, browser UI smoke, playtest smoke, and documentation validation. Documentation is part of milestone quality now: PDFs must be regenerated, tables of contents must be visible in source guide/reference pages, and screenshots must be current and task-specific.

Current Limitations

- full CRUD coverage is still incomplete across all object classes
- editor information architecture cleanup is still future work

- richer player profile and front-end state presentation are still future work
- no true runtime map-window scrolling yet for oversized maps
- starter-library and graphics completion are still future work
- no cloud or account system yet
- no multiplayer yet
- no end-user AI authoring workflow yet

Future Roadmap Alignment

- `32`: AI game creation from prompt. `32A` now maps create, finish, and expand game prompts onto the shared AI generation/review contract; remaining work adds one real provider adapter, provider configuration, model selection, request submission, and reviewed proposal preview inside the application
- `33`: optional AI-driven NPC behavior and shared actor-capable runtime growth
- `34`: multiplayer
- `35`: mobile play-only shell
- `36`: editor completion and editor-skinning separation
- `37`: player front end and runtime-skinning separation, including runtime map-window scrolling
- `38`: renderer family choice, including renderer-family viewport rules for large maps
- `39`: starter libraries and graphics completion
- `40`: WorldTree naming and compatibility migration

Subject Glossary

Product Terms

- `Play Mode`: the playable runtime
- `Edit Mode`: the authoring environment
- `Test & Publish`: the release and export area inside the editor

Authoring Terms

- `AdventurePackage`: top-level authored content object
- `Draft`: mutable editor state
- `Project`: mutable backend-backed editable record

- `Release`: immutable published snapshot

Runtime Terms

- `RuntimeSnapshot`: serializable runtime state
- `GameSession`: runtime orchestration object
- `EngineEvent`: runtime event emitted during play

Publishing Terms

- `Forkable package`: editable release-backed handoff
- `Standalone package`: play-only release-backed handoff
- `Artifact integrity report`: parity check across release handoffs

AI Terms

- `Provider manifest`: provider identity and capability description
- `Proposal`: structured AI-generated content candidate
- `Review report`: normalized readiness and issue summary
- `Application plan`: safe-apply summary
- `Import dossier`: reviewed AI handoff import package

Technical Catalog

- `apps/web/src/index.ts`: browser runtime orchestration
- `apps/web/src/editor.ts`: browser editor orchestration
- `apps/api/src/index.ts`: local project/release API
- `apps/api/src/standalone-bundle.ts`: standalone bundle assembly
- `packages/runtime-core/src/index.ts`: runtime rules exports
- `packages/editor-core/src/index.ts`: editor mutation exports
- `packages/validation/src/index.ts`: shared validation
- `packages/publishing/src/index.ts`: release-backed artifact exports
- `packages/ai-core/src/index.ts`: AI review contract exports

Diagrams Appendix

The long-term reference set should include package boundary, runtime flow, editor flow, publishing flow, AI review lifecycle, and skinning separation diagrams.

Change Log And Reference Maintenance

This reference should be updated whenever accepted planning or implementation changes alter major system flows, package responsibilities, publishing behavior, AI integration behavior, skinning boundaries, or documentation standards.

Maintenance rule from this point forward: the System Reference should stay readable top-down, introduce concepts before file-level detail, and remain aligned with the roadmap, User Guide, and AI-readable project context.